Universität Konstanz

Introduction to Computing for the Social Sciences

Session 2 – Information Coding

David Garcia

Winter Semester 2026-2027

Course Outline

- *Session 1: Introduction*
- **Session 2: Information Coding**
- Session 3: Data Structures
- Session 4: Software
- Session 5: Algorithms
- Session 6: Recursion
- Session 7: Sorting Algorithms
- Session 8: Time Complexity
- Session 9: Theoretical Computer Science
- Session 10: Machine Learning
- Session 11: Databases
- Session 12: Parallel Programming
- Session 13: Social Science Applications
- Session 14: Exam Review

Outline

1. Information Coding
2. Conversion Between Number Coding Systems
3. Character Coding
4. Python Data Types and Binary Operations

Information coding matters (I)

Universität Konstanz

EXCEL RENAMING GENES

An audit of 3,597 published genetics papers found that **704 of them, one in five, contained corrupted gene names.** The spreadsheet had read `SEPT2` as a date and stored it as `2-Sep`; identifiers like `2310009E13` became `2.31E+13`.

- **20%** of papers affected
- **0** warnings shown
- **27** genes renamed in 2020 so the software would stop breaking them

What is a spreadsheet actually storing when it shows you `SEPT2`, and who decided what it means?

Ziemann, Eren & El-Osta, *Genome Biology* (2016); Bruford et al., *Nature Genetics* (2020)

Information coding matters (II)

Universität Konstanz

THE STOCK INDEX THAT LOST HALF OF ITS VALUE

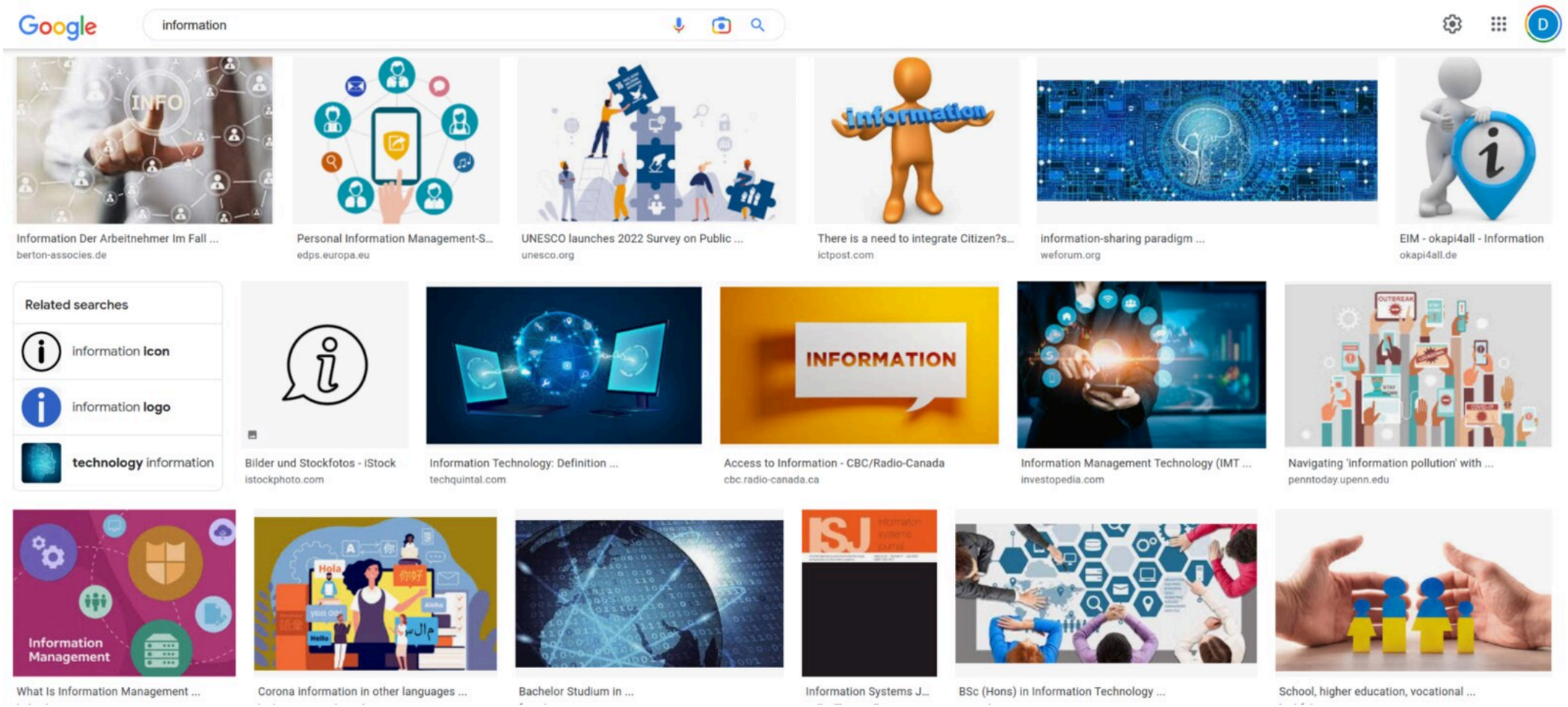
The Vancouver Stock Exchange index opened at **1000.000** in 1982 and was recomputed after every trade. Months later it read **524.811**, while the shares it tracked had risen. Each recomputation **truncated** the fourth decimal instead of rounding, thousands of times a day.

- **1000.000:** when launced launch in Jan 1982
- **524.811:** value displayed in Nov 1983
- **1098.892:** true value, once recomputed

Why can’t a computer store 0.1 exactly, and what does that cost after a million additions?

Toronto Star, 29 Nov 1983; discussed in Higham, *Accuracy and Stability of Numerical Algorithms*

What is information?



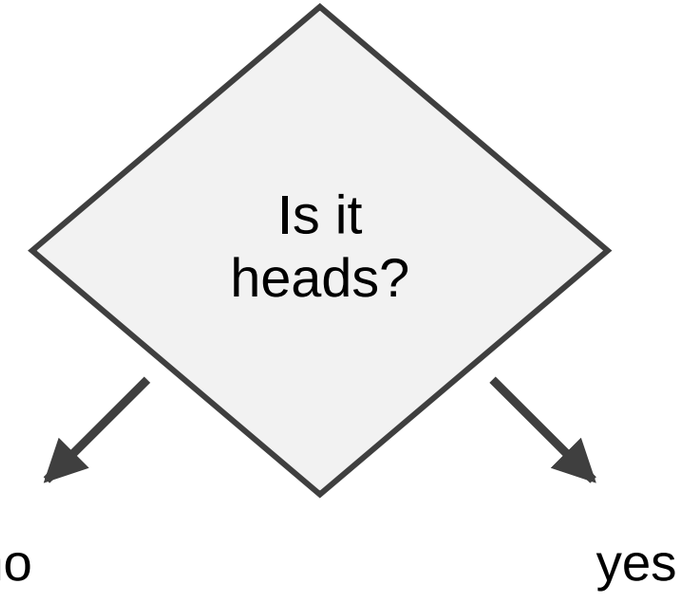
In Computer Science, information is the representation of knowledge

Encoding information

Information is encoded by recording changes in knowledge.

For example, what we learn from the answer to a question



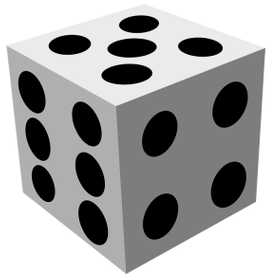


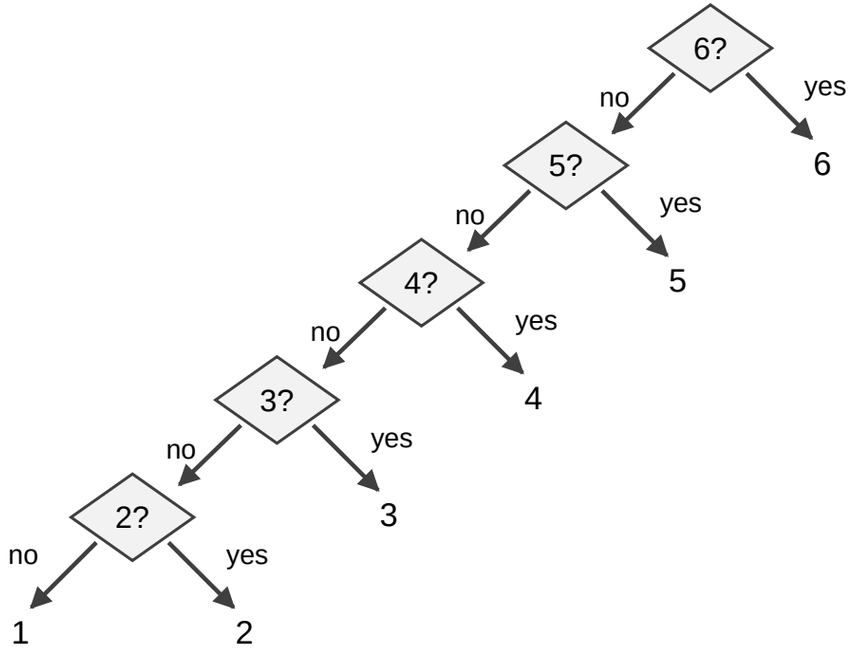
Measuring information in bits

Each yes/no question gives us a **bit** of information.

How many bits do we learn from the result of a dice throw?

This depends on how we ask to find out the dice result, i.e. **how we encode the information**



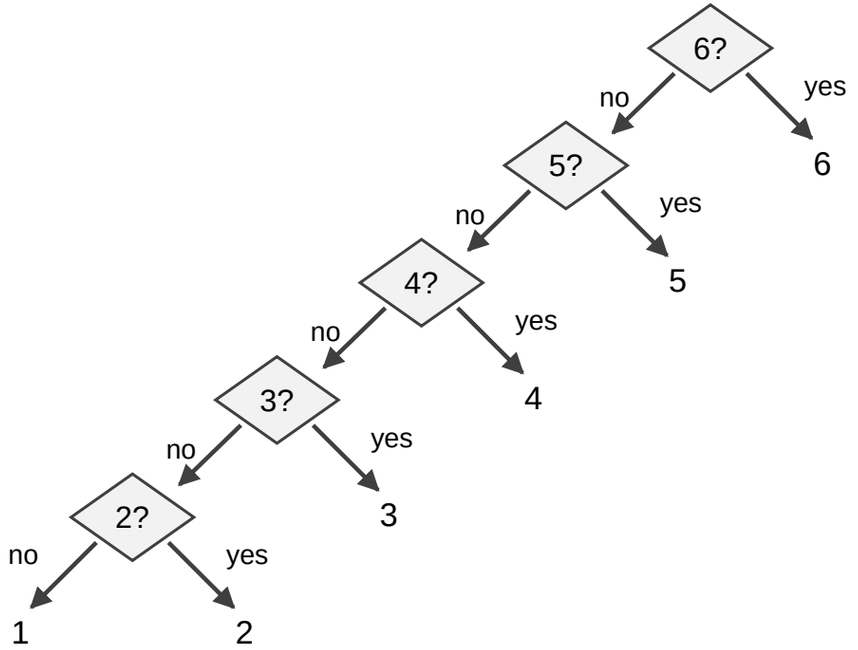


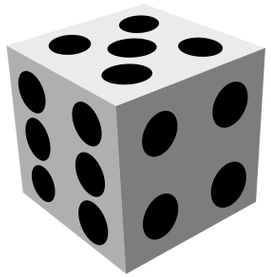
Measuring information in bits

Each yes/no question gives us a **bit** of information.

How many bits do we learn from the result of a dice throw?

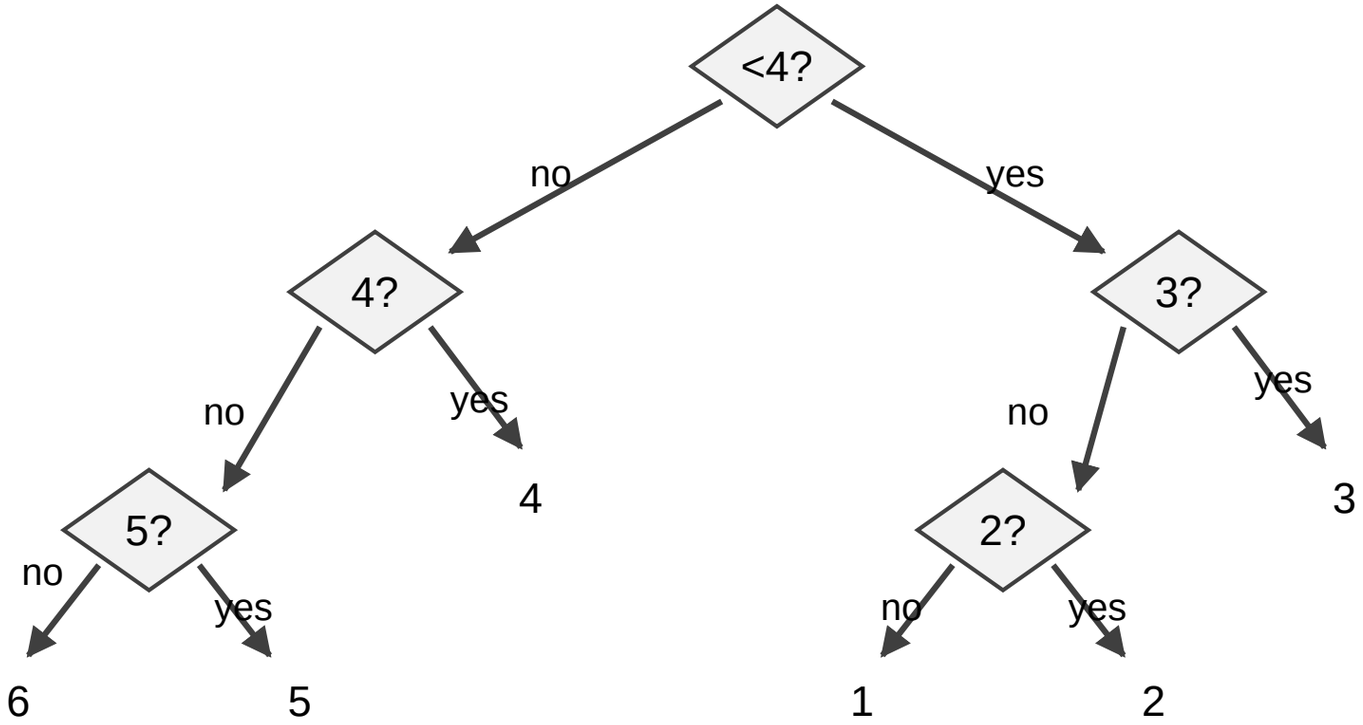
This depends on how we ask to find out the dice result, i.e. **how we encode the information**





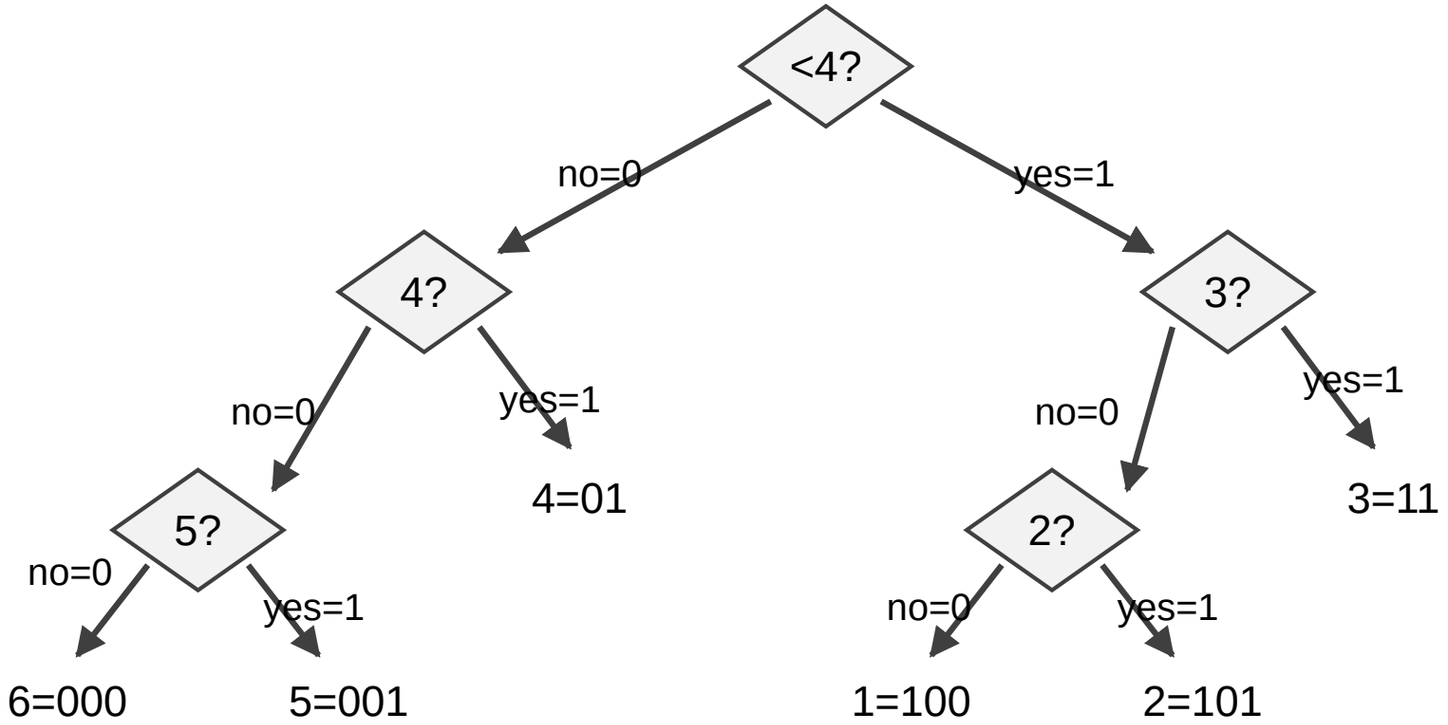
On average we will need to ask 20/6 questions (3.33). Can we do better?

Encoding information more efficiently



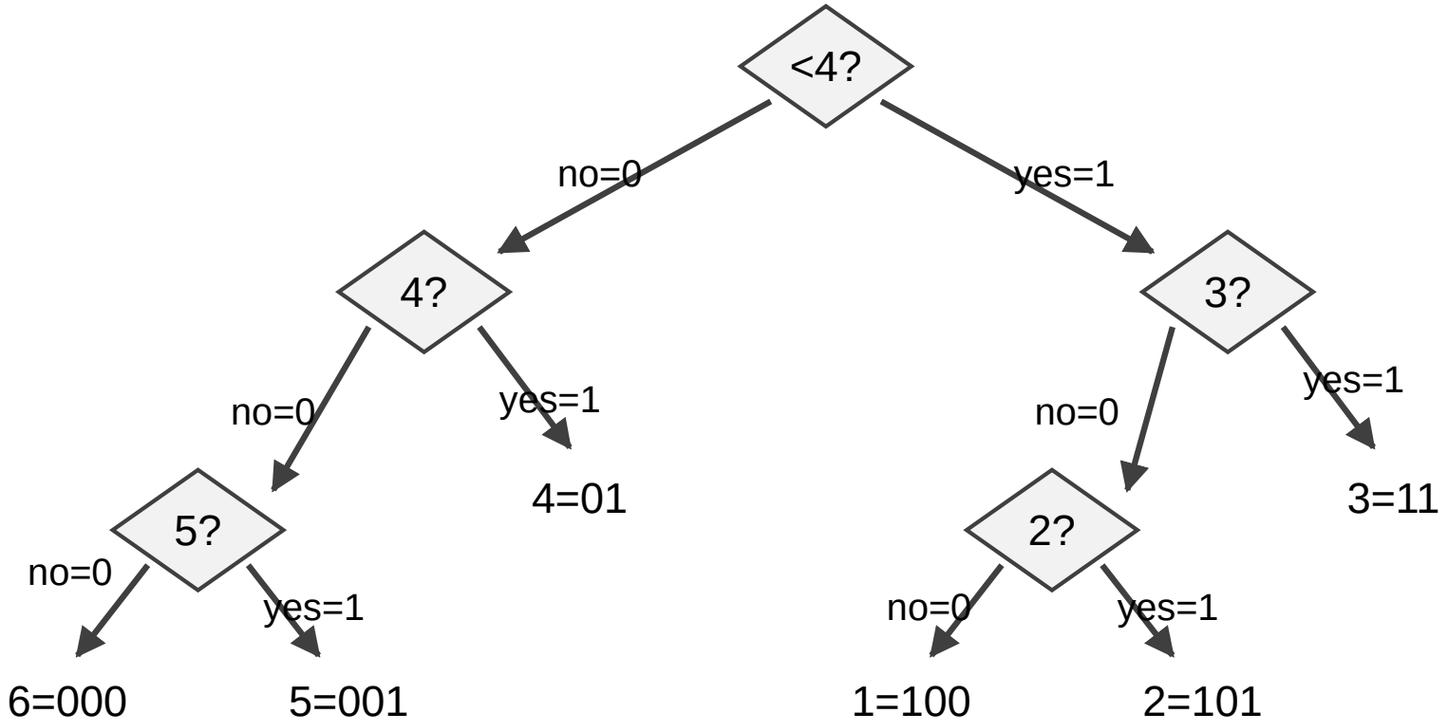
- This way we only need 16/6 questions on average (2.66)
- Each yes/no question doubles the number of possibilities,
  - e.g. a result with 8 equally likely possibilities has 3 bits of information

Encoding information more efficiently



- Encoding the result of a dice (1-6) with three bits (0 or 1)
- **Prefix code**: no coding is a prefix of another, allows unambiguous encoding

Decoding information

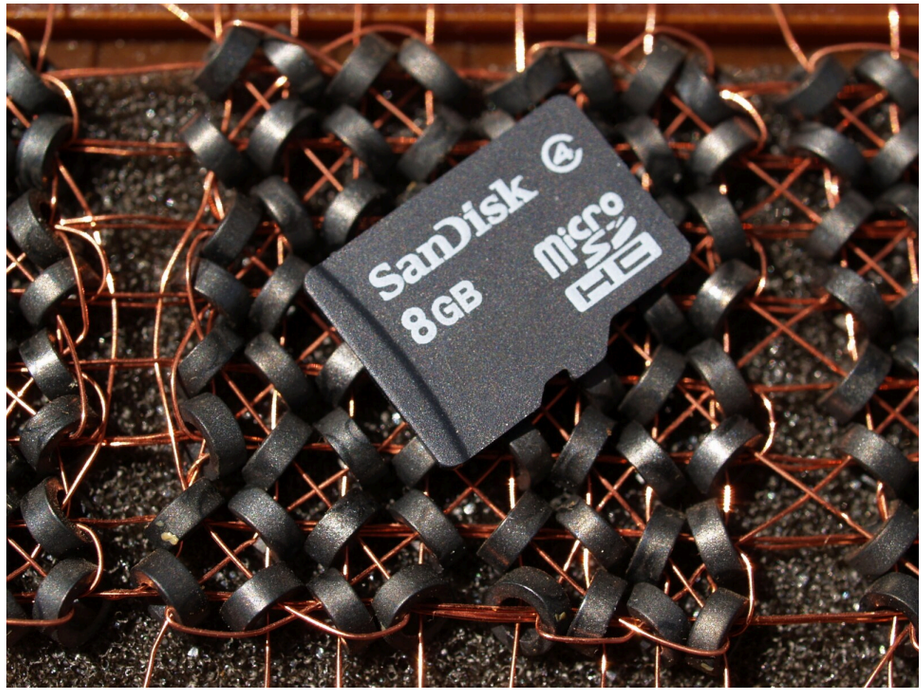


- To decode, we translate codes back to the values they mean
- For example, if we encoded the following bits, what were the results of the dice?

```
1101000100101
```

Storing information digitally

- Smallest physical information unit is 1 **bit**
  - no/yes is represented as 0/1
  - memory: a capacitor or a capacitor and several transistors
  - hard disk: magnetization

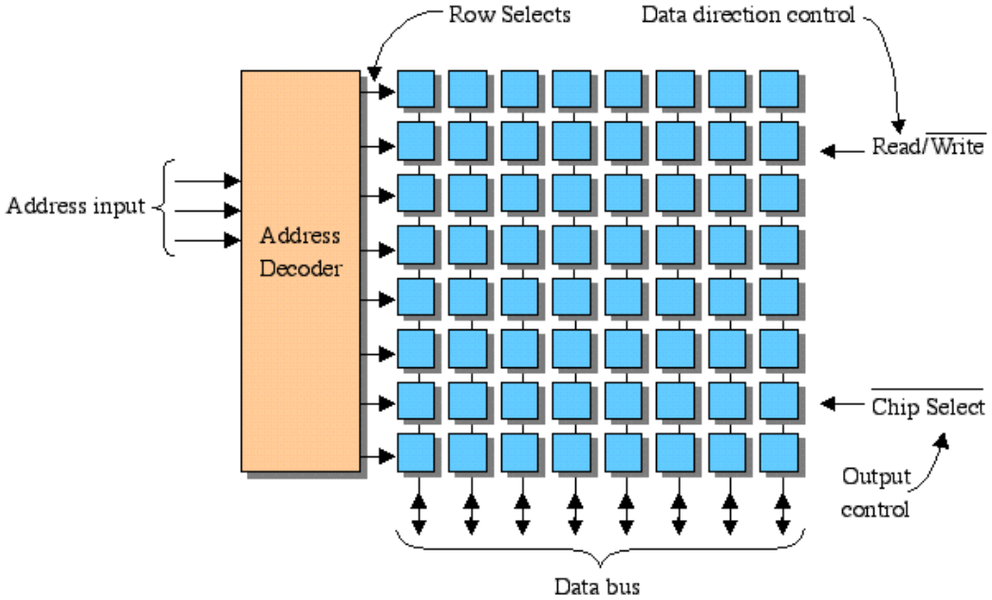


- Smallest logical unit is 1 **byte**: 8 bit
  - *logical* means that it can be accesses with a memory address and operated on
  - processors can usually only address entire bytes, i.e. information in chunks of 8 bits

How memory works

- Bits in chips are encoded as voltage values (e.g. 0 = 0 Volts, 1 = +5 Volts)
- Memory chips have three inputs
  - A line called “select” to activate
  - A line to specify read/write
  - An address line with several bits to select the row where to read/write
- The address decoder activates the corresponding row of the chip
- The data bus outputs the content of the row (read case) or contains the new value (write case)





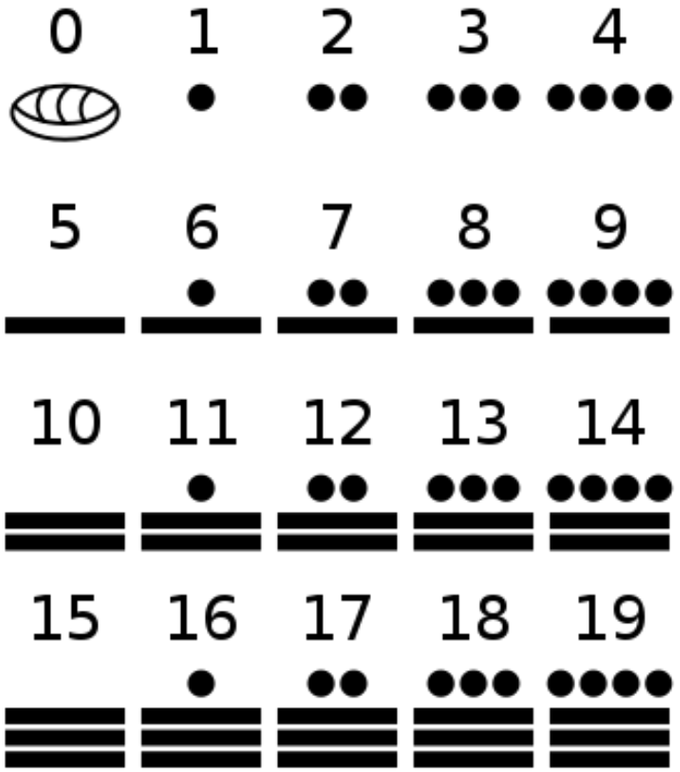
Tarnoff, ETSU: computer memory

Conversion Between Number Coding Systems

1. Information Coding
2. **Conversion Between Number Coding Systems**
3. Character Coding
4. Python Data Types and Binary Operations

Positional number systems

- Value of digit depends on its position
- Composed of a finite number of numeric symbols (a.k.a. alphabet)
- Each position represents a specific value base^k (base^3 base^2 base^1 base^0)
- Examples:
  - Decimal number system:
    - In 12, digit 1 has value 10
    - in 1200 digit 1 has value 1000
  - Unary number system: to represent number N, an arbitrary symbol representing 1 is repeated N times
- Positional number systems require a symbol for zero.
  - e.g. the Mayas are a base-20 positional number system



Wikipedia: Maya numerals

The base of a positional number system

- Base is also often referred to as **radix**
  - base 10: decimal notation (0 – 9)
  - base 2: binary notation (0, 1)
  - base 8: octal notation (0 – 7)
  - base 16: hexadecimal notation (0 – 9, A – F)

- Notation convention is to indicate the base, if not clearly known, as subscript
  - $7_{10} = 7_8 = 111_2$
  - $9_{10} = 11_8 = 1001_2$
  - $14_{10} = 16_8 = 1110_2$

Conversions Between Systems: Informal Example

- Which is greater: 47_8 or 43_{10} ?

Conversions Between Systems: Informal Example

- Which is greater: 47_8 or 43_{10} ?

- Convert from octal to decimal notation

$$47_8 = 7 \times 8^0 + 4 \times 8^1 = 39_{10}$$

Conversions Between Systems: Informal Example

- Which is greater: 47_8 or 43_{10} ?

- Convert from decimal to octal notation

$$43_{10} = 3 \times 8^0 + 5 \times 8^1 = 53_8$$

Informal examples

Decimal notation

- $5648_{10} = 8 \times 10^0 + 4 \times 10^1 + 6 \times 10^2 + 5 \times 10^3$

Octal to decimal

- $1346_8 = 6 \times 8^0 + 4 \times 8^1 + 3 \times 8^2 + 1 \times 8^3 = 742_{10}$

Hexadecimal to decimal

- $\mathrm{CB6}_{16} = 6 \times 16^0 + 11 \times 16^1 + 12 \times 16^2 = 3254_{10}$

How can we do this formally for any base?

Conversions Between Number Systems: Algorithm

Universität Konstanz

- Conversion of a **decimalinteger** $n \geq 0$ **to binary** notation

Procedure DECIMAL-TO-BINARY(n)
Inputs:
n: a positive integer written in decimal
Result: the binary digits of n, written as $x_{k-1} \ldots x_1 x_0$
$k \leftarrow 0$
While $n > 0$:
    $x_k \leftarrow n$ **mod** 2
    $n \leftarrow \lfloor n/2 \rfloor$
    $k \leftarrow k + 1$
return $x_{k-1} \ldots x_1 x_0$

Example

Division	Quotient	Remainder
190 : 2 =	95	0
95 : 2 =	47	1
47 : 2 =	23	1
23 : 2 =	11	1
11 : 2 =	5	1
5 : 2 =	2	1
2 : 2 =	1	0
1 : 2 =	0	1

$$190_{10} = 1011\,1110_2$$

Conversions Between Number Systems: Algorithm

- Generalization: conversion of a **decimal integer** $n \geq 0$ **to base-**β notation

Procedure DECIMAL-TO-BASE(n, β)

Inputs:

n: a positive integer written in decimal

β: the target base

Result: the digits of n in base β, written as $x_{k-1} \ldots x_1 \, x_0$

$k \leftarrow 0$

While $n > 0$:

    $x_k \leftarrow n$ **mod** β

    $n \leftarrow \lfloor n/\beta \rfloor$

    $k \leftarrow k + 1$

return $x_{k-1} \ldots x_1 \, x_0$

Converting between power of 2 bases

- Conversion of **octal to binary** notation
  - every digit 0 - 7 can be represented exactly by 3 binary digits
    - allows for elegant and fast conversion digit by digit using blocks of 3 binary digits

Examples

- $47_8 = 39_{10} = ?_2$

Converting between power of 2 bases

- Conversion of **octal to binary** notation
  - every digit 0 – 7 can be represented exactly by 3 binary digits
    - allows for elegant and fast conversion digit by digit using blocks of 3 binary digits

Examples

- $47_8 = 39_{10} = 1 \times 2^0 + 1 \times 2^1 + 1 \times 2^2 + 0 \times 2^3 + 0 \times 2^4 + 1 \times 2^5 = 100\,111_2$

Converting between power of 2 bases

- Conversion of **octal to binary** notation
  - every digit 0 – 7 can be represented exactly by 3 binary digits
    - allows for elegant and fast conversion digit by digit using blocks of 3 binary digits

Examples

- $47_8 = 100\,111_2$
- $134_8 = ?_2$
- $460231_8 = ?_2$

Oct	0	1	2	3	4	5	6	7
Binary	000	001	010	011	100	101	110	111

Converting between power of 2 bases

- Conversion of **binary to octal** notation
  - divide binary digits into groups of three bits and convert each group of bits to an octal digit

Examples

- $101\,011_2 = 53_8$
- $010\,011\,100\,101\,111_2 = 23457_8$

Oct	0	1	2	3	4	5	6	7
Binary	000	001	010	011	100	101	110	111

Converting between power of 2 bases

Universität Konstanz

- Conversion of **hexadecimal to binary** notation (also noted with `0x` prefix)
  - every digit 0 – 9 and letters A – F can be represented by exactly 4 binary digits
  - allows for elegant and fast conversion digit by digit using blocks of 4 binary digits

Examples

- `0x23A7` $= 23\mathrm{A}7_{16} = 0010\,0011\,1010\,0111_2$
- `0xB3D2` $= \mathrm{B}3\mathrm{D}2_{16} = 1011\,0011\,1101\,0010_2$

Hex	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
Binary	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111

Converting decimal numbers

Universität Konstanz

- Conversion of a number n **with only decimal places to binary** notation
  - notation of a binary number with decimal places is $0.x_{-1}x_{-2}\ldots x_{-k}$

Procedure FRACTION-TO-BINARY(n)

Inputs:

n: a number written in decimal, with $0 < n < 1$

Result: the binary digits of n, written as $0.x_{-1}\ x_{-2}\ \ldots\ x_{-(k-1)}$

$k \leftarrow 1$

While $n > 0$:

  $x_{-k} \leftarrow \lfloor 2n \rfloor$

  $n \leftarrow 2n - x_{-k}$

  $k \leftarrow k + 1$

return $0.x_{-1}\ x_{-2}\ \ldots\ x_{-(k-1)}$

Example:

		Value
0.34375	× 2 = 0.6875	0
0.6875	× 2 = **1**.375	1
0.375	× 2 = 0.75	0
0.75	× 2 = **1**.5	1
0.5	× 2 = **1**.0	1

$$0.34375_{10} = 0.01011_2$$

Negative Binary Numbers: One's Complement

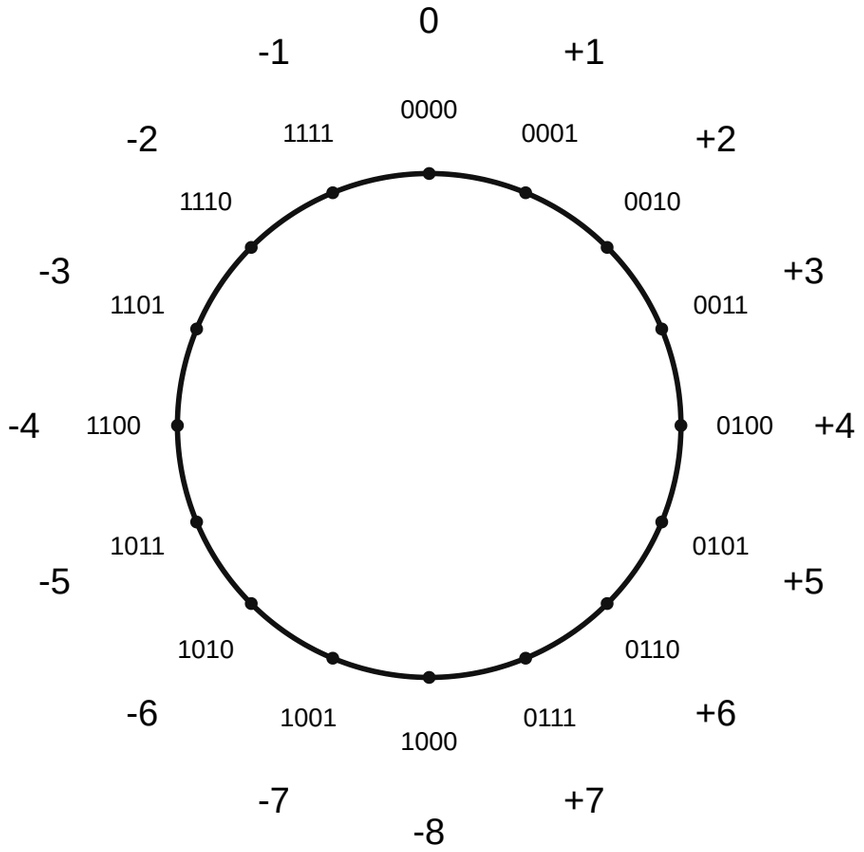
- **Ones' Complement**
  - leading bit codes the sign (0: +, 1: –)
  - $k-1$ digits to represent the absolute value of a given number
  - negation by inverting of the number
    - bit-wise flipping of 1s and 0s
  - Problem: there are two representations of 0:
    - `0000 0000`
    - `1111 1111`
  - Why is this a problem?

Example

- $\underline{0}001\,1011_2 = 27_{10}$
- $\underline{1}110\,0100_2 = -27_{10}$

Negative Binary Numbers: Two's Complement

- **Two's Complement**
  - leading bit codes the sign (0: +, 1: –)
  - negation by:
    1. flipping the bits of the number
    2. adding one
  - unique representation of 0: `0000 0000`



Example

$$27_{10} = 0001\,1011_2$$
$$1110\,0100_2 \quad \text{(inverted)}$$
$$1110\,0101_2 = -27_{10}$$

Overflow

- **When an operation causes a value to go beyond the limits of its encoding**

Example in 2's complement:

$$
\begin{array}{ll}
0111\,1111 & (127_{10}) \\
+ & \\
0000\,0111 & (7_{10}) \\
1000\,0110 & (134_{10}? \text{ No! } -122_{10})
\end{array}
$$

- Other typical example is division by zero
- Operating system or programming language throws a warning or an error.

judge: I sentence you to the maximum punishment...

Me. May I have one day more?

judge: Ok, you have to serve −32.768 years in prison.

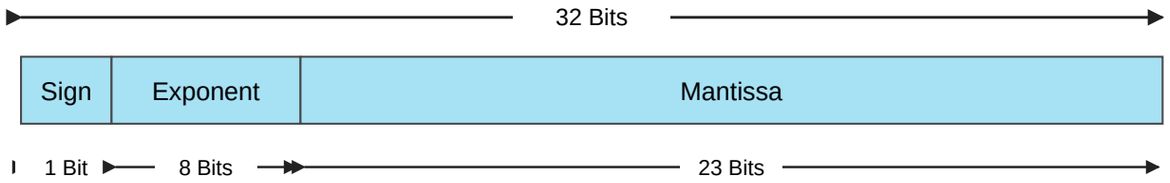


Floating-Point numbers: IEEE 754

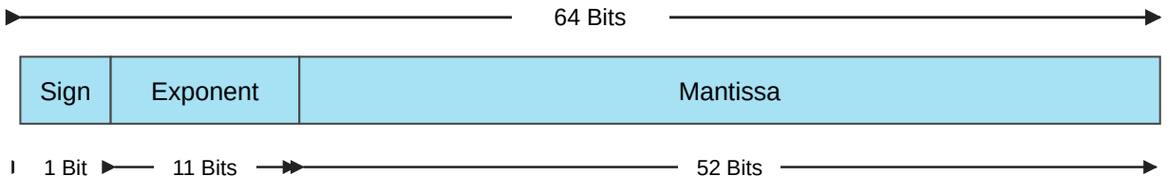
- Standardized according to IEEE 754:
  - single or double precision (32 or 64 bit, `float` or `double`)
  - 1 bit for the **sign of mantissa**: $s = (-1)^S$
  - 8/11 bit **biased exponent**: $e = E - B$
    - bias permits to have a longer range of negative exponents
    - $B = 127$ or $B = 1023$
  - 23/52 bit **normalized mantissa** with implicit 1: $m = 1.M$
    - leading 1 is not explicitly stored
    - mantissa and exponent are adjusted accordingly (normalized)

Floating-Point numbers: IEEE 754

- Standardized according to IEEE 754:
  - single or double precision (32 or 64 bit, `float` or `double`)
  - 1 bit for the **sign of mantissa**: $s = (-1)^S$
  - 8/11 bit **biased exponent**: $e = E - B$
  - 23/52 bit **normalized mantissa** with implicit 1: $m = 1.M$

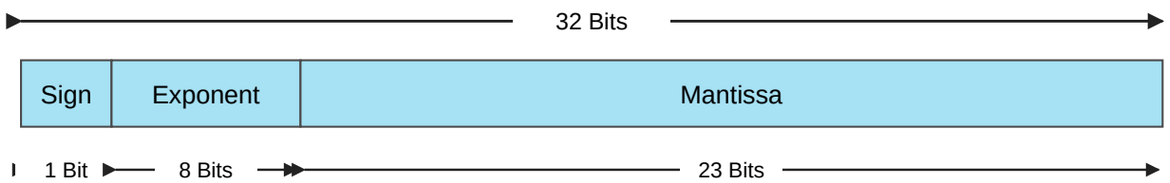


Single Precision
IEEE 754 Floating-Point Standard

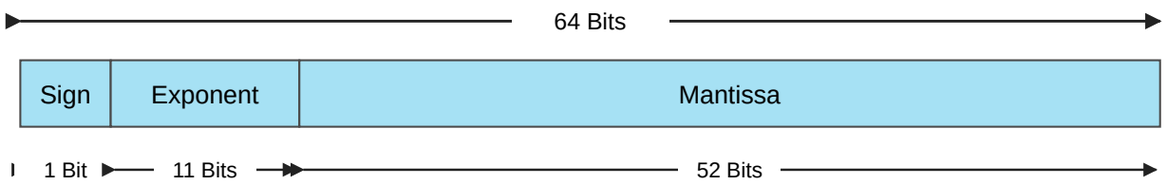


Double Precision
IEEE 754 Floating-Point Standard

Floating-Point numbers: IEEE 754



Single Precision
IEEE 754 Floating-Point Standard



Double Precision
IEEE 754 Floating-Point Standard

- Special representation for
  - 0 : exponent and mantissa are both 0
    - there is a positive and a negative 0
  - infinity : exponent E only 1s, mantissa $M = 0$
- Value ranges: single precision: $1.4 \cdot 10^{-45} \leq |z| \leq 3.4 \cdot 10^{38}$
- double precision: $4.9 \cdot 10^{-324} \leq |z| \leq 1.7 \cdot 10^{308}$

Floating-Point numbers: IEEE 754

Example: Decimal value to float value

18.625 in single precision

- bias $B = 127 = 2^7 - 1$
- transform to binary notation: $18.625_{10} = 10010.101000\ldots_2$
- normalize to calculate exponent: $10010.101000\ldots_2 = 1.0010101000 \cdot \cdots \cdot 2^4$
- exponent with bias: $E = 4_{10} + 127_{10} = 131_{10} = 1000\,0011_2$
- sign $S = 0$
- result: `0|10000011|00101010000000000000000`

Floating-Point numbers: IEEE 754

Example: Float value to decimal value

`1|10010011|10100000000000000000000` in single precision

- sign $S = 1$, i.e., a negative number
- exponent $E = 147 = 127 + 20$, i.e., $e = 20$
- mantissa $m = 1 + 1 \cdot 2^{-1} + 1 \cdot 2^{-3} = 1.625$
- result
  - $-1.625 \cdot 2^{20} = -1\,703\,936.0$

Numerical formats matter in AI

AI models require a lot of memory in GPU (VRAM). Models are based on parameters that are often fitted as floating-point numbers.

Common formats in AI:

- FP32 (32-bit Floating-Point)
- FP16 (16-bit Floating-Point / Half-Precision)
- BF16 (bfloat16 / Brain Floating Point)
- FP8 (8-bit Floating-Point)
- INT8 (8-bit Integer, for Quantized Models)
- INT4 / INT2 (Quantized, Low-Bit Formats)

Model Size	FP16	FP8	INT4
8B	16 GB	8 GB	4 GB
70B	140 GB	70 GB	35 GB
405B	810 GB	405 GB	203 GB

- Llama 3.1 – 405B, 70B & 8B memory requirements by precision
- Lower precision affects model performance

Underflow: Rounding errors in Floating-Point

- Limited value range
  - can not represent every possible number
  - in practice double precision does cover most cases though
- Limited precision, rounding errors
  - limited room for decimal places
  - certain decimal numbers can not be expressed precisely in binary notation:
    - very common example: 0.1
  - accumulation of small rounding errors possible

Rounding error example

```
1  f = 2
2  for i in range(1, 100):
3      f += 10 ** (-1)
4      print(f)
```

```
>> 2.1
>> 2.2
>> 2.3000000000000003
>> 2.4000000000000004
```

Rounding and floating point errors



- Gulf War, 1991: Iraqi Scud missile hit American barracks in Dhahran, Saudi Arabia, killing 28 soldiers and wounding 260.
- “A 24-bit representation of 0.1 was used to multiply the clocktime, yielding a result in a pair of 24-bit registers. This was transformed into a 48-bit floating-point number.”
- “Roundoff Error and the Patriot Missile”, Robert Skeel, SIAM News (1992)



- Ariane 5 maiden flight, 1996: Rocket exploded 39 seconds after liftoff.
- Error when converting horizontal velocity from a 64-bit floating point number to a 16-bit signed integer between the software of Ariane 5 and Ariane 4.
- N° 33–1996: Ariane 501 - Presentation of Inquiry Board report

COBOL and Grace Hopper

- COBOL is a very old programming language, designed in 1959 but still very used in banking, finance, and government
- COBOL does no rounding by default on fractions, minimizing the risk of rounding errors
- Grace Hopper, a.k.a. grandma COBOL, was an American military computer scientist that invented machine-independent programming languages, COBOL being one of the first



https://www.tpr.org/technology-entrepreneurship/2019-05-23/how-cobol-still-powers-the-global-economy-at-60-years-old

Character Coding

1. Information Coding
2. Conversion Between Number Coding Systems
3. **Character Coding**
4. Python Data Types and Binary Operations

ASCII – Code

Universität Konstanz

- *American Standard Code for Information Interchange*
- Standard since 1963, still in use today

	0	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**A**	**B**	**C**	**D**	**E**	**F**
0	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	BS	HT	LF	VT	FF	CR	SO	SI
1	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS	RS	US
2	SP	!	"	#	$	%	&	'	(	)	*	+	,	-	.	/
3	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	DEL

- initially 7 bit code for tele printers, all codes of US keyboard in one byte
  - 128 characters, 33 control characters, 95 printable characters
- Missing characters for other keyboards

Code Pages

- Extension of ASCII-Codes to 8 bit (1 byte)
- CP850, CP437, ISO-8859-1 to ISO-8859-15 with characters for other languages

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	BS	HT	LF	VT	FF	CR	SO	SI
1	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS	RS	US
2		!	"	#	$	%	&	'	(	)	*	+	‚	-	·	/
3	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	–
6	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	DEL
8	PAD	HOP	BPH	NBH	IND	NEL	SSA	ESA	HTS	HTJ	VTS	PLD	PLU	RI	SS2	SS3
9	DCS	PU1	PU2	STS	CCH	MW	SPA	EPA	SOS	SGCI	SCI	CSI	ST	OSC	PM	APC
A	NBSP	¡	¢	£	€	¥	Š	§	š	©	ª	«	¬	SHY	®	¯
B	°	±	²	³	Ž	µ	¶	·	ž	¹	º	»	Œ	œ	Ÿ	¿
C	À	Á	Â	Ã	Ä	Å	Æ	Ç	È	É	Ê	Ë	Ì	Í	Î	Ï
D	Ð	Ñ	Ò	Ó	Ô	Õ	Ö	×	Ø	Ù	Ú	Û	Ü	Ý	Þ	ß
E	à	á	â	ã	ä	å	æ	ç	è	é	ê	ë	ì	í	î	ï
F	ð	ñ	ò	ó	ô	õ	ö	÷	ø	ù	ú	û	ü	ý	þ	ÿ

Unicode

- In ASCII, not all possible characters can be represented, e.g., kanji, chinese
- Solution: development of standardized **Unicode**
  - simultaneous representation of all possible characters
  - version 1.0 in 1991 with a 16-bit code space (65 536 code points)
  - 2022 version (15.0) has 149,186 characters
  - Notation: `U+` and at least 4 hexadecimal numbers
    - e.g.: `U+00DF` = ß

Representation of Unicode

- 3 bytes are necessary for all possible characters
  - wasting memory in many geographical regions, e.g. Europe
- Definition of **Unicode Transformation Formats (UTF)** as solution
- **UTF-16**
  - 2 byte per characters but still wastes memory
  - other areas are covered by combination of UTF-16 characters
- **UTF-8**
  - 1 byte per character, covers most important Western characters
  - Compatible with ASCII Codes
  - other characters use a sequence of up to four bytes

Usual Character Encoding Issues

Universität Konstanz

- Character encoding is a usual source of problems with social science data
- Try to control coding in documentation and keep it general (e.g. UTF-8)

```
>>> u = chr(40960) + 'abcd' + chr(1972)
>>> u.encode('utf-8')
b'\xea\x80\x80abcd\xde\xb4'
>>> u.encode('ascii')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
UnicodeEncodeError: 'ascii' codec can't encode character '\ua000' in position 0:
    ordinal not in range(128)
>>> u.encode('ascii', 'ignore')
b'abcd'
>>> u.encode('ascii', 'replace')
b'?abcd?'
>>> u.encode('ascii', 'xmlcharrefreplace')
b'&#40960;abcd&#1972;'
```

Python Data Types and Binary Operations

1. Information Coding
2. Conversion Between Number Coding Systems
3. Character Coding
4. **Python Data Types and Binary Operations**

Primitive Data Types

Universität Konstanz

Type	Bits	Coding	Minimum	Maximum
`boolean`	1/8	truth value	false	true
`byte`	8	two’s complement	-128	127
`short`	16	two’s complement	-32 768	32 767
`int`	32	two’s complement	-2 147 483 648	2 147 483 647
`long`	64	two’s complement	-9 223 372 036 854 775 808	9 223 372 036 854 775 807
`char`	8-32	UTF-8 (standard)	0 (U+0000)	1 114 111 (U+10FFFF)
`float`	32	IEEE 754		
`double`	64	IEEE 754		

These are the primitive types of languages like C and Java. Python hides them behind `bool`, `int` (unbounded), `float` (a 64-bit `double`) and `str` (Unicode text).

Type Conversion

Universität Konstanz

- Expressions can be turned into other types
  - general rules are:
    - conversion to a “larger” data type can be done without loss of precision
    - conversion to a “smaller” data type usually leads to loss of precision
    - conversion of a floating point number to an integer cuts the decimal places
  - general type hierarchy:
    - `byte` ⊂ `short` = `char` ⊂ `int` ⊂ `long`
    - `float` ⊂ `double`
- Python “hides” most of this complexity
  - you can, for example, add or multiply numbers of different type and the number formats are correctly managed

Automatic Type Conversion

- Automatic type conversion is especially important for arithmetic operations

- binary operators are only defined for the same data type:
  - `double` + `int`, requires conversion of `int` to `double`
    e.g. $10.5 + 5 = 15.5$
  - `double` + `float`, requires conversion of `float` to `double`
    e.g. $10.5f + 5.2 = 15.7$
  - resulting type is always the “larger” type

Operators

- Simple mathematical operations `+`, `-`, `*`, `/`
- Comparison operators `<`, `>`, `<=`, `>=`, `==`, `!=`
- Brackets for mathematical operations
- Modulo operator `%`
  - `a % b` yields the remainder of the integer division of `a` by `b`
- Logical operators (only for truth values)
  - `not`: logical negation
  - `and`: logical and (True and True = True, else False)
  - `or`: logical or (False or False = False, else True)
  - `^`: exclusive or (`False ^ True = True`, `True ^ False = True`, else `False`)

Operators

Universität Konstanz

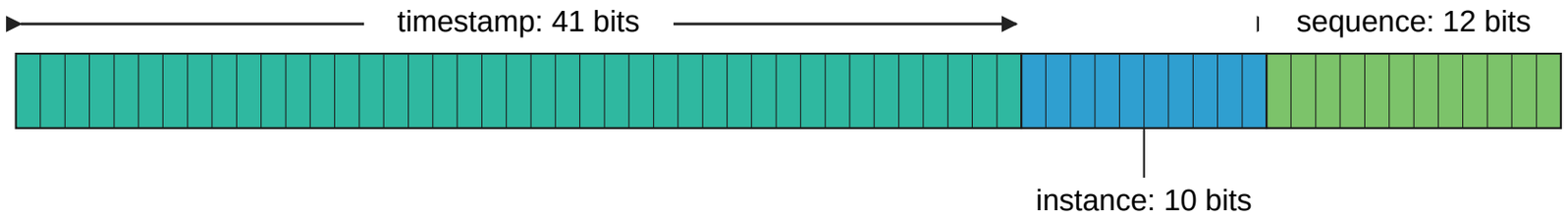
- Bitwise operators (only for integers)
  - `~` bitwise complement: flips every bit (in two's complement `~x = -x - 1`)
  - `&` bitwise and (`1 & 1 = 1`, else 0)
  - `|` bitwise or (`0 | 0 = 0`, else 1)
  - `^` bitwise exclusive or (`1^0 = 1`, `0^1 = 1`, else 0)
  - `<<` bitwise shift to the left (`0010 << 2 = 1000`)
  - `>>` bitwise shift to the right (`0110 >> 2 = 0001`)

Python Refresh: Try yourself with some binary or hexadecimal operations

Social Science Application: the secret of Twitter snowflakes

"id":

1076543210012345678 9




```
1  def get_tweet_timestamp(tid):
2      offset = 1288834974657
3      tstamp = (tid >> 22) + offset
4      utcdttime = datetime.utcfromtimestamp(tstamp/1000)
5      print(str(tid) + " : " + str(tstamp) + " => " + str(utcdttime))
```

https://github.com/oduwsdl/tweetedat

https://ws-dl.blogspot.com/2019/08/2019-08-03-tweetedat-finding-tweet.html

Summary

- Information answers a question: one bit is one yes/no answer, and frequent outcomes deserve shorter codes
- Computers store everything as bits; binary, octal and hexadecimal are ways to write them
- Negative integers use two’s complement; fixed-width integers can overflow
- IEEE 754 floating point approximates real numbers, so rounding errors appear and add up
- Text needs a character encoding (ASCII, Unicode, UTF-8); reading with the wrong one corrupts it
- Data types decide what the bits mean and which operations apply

Next session: Data Structures